

CASA Evidence — Inventory of APIs Accepting User-Controlled Input

Question: Provide a list of APIs in which portions of the API/URL or parameters may be passed from the user into the application.

Application: Provenance (Next.js App Router monorepo). Two API surfaces exist:

Provenance's own frontend, session-cookie authenticated.

— a separate deployable app, Bearer-API-key authenticated, intended for third-party integrators (certificate verification, asset lookup).

- **Internal application API** (`src/app/api/**/route.ts`) — called by
- **Public partner REST API** (`apps/api/src/app/api/v1/**/route.ts`)

For every route below, "user-controlled input" means one or more of: a **dynamic URL path segment** (e.g. `[id]`), a **query-string parameter**, or a **request body** (JSON or `multipart/form-data`). This document also lists dynamic **page** routes, since the question covers "API/URL" broadly, and notes the common input-validation controls applied across the whole surface (§4).

1. Public Partner REST API (`apps/api` — `/api/v1/*`)

All routes below require **Authorization: Bearer <api_key>** (`authenticateRequest()` in `apps/api/src/middleware/auth.ts`), which looks the key up by SHA-256 hash, checks `is_active/expires_at`, and enforces a per-key hourly rate limit (Upstash-backed). Several routes additionally enforce **scope** and **planet** restrictions baked into the API key itself.

Endpoint	Method	User-controlled input	Auth / authorization	Validation
<code>/api/v1/verify</code>	POST	Body: <code>planet</code> (or legacy alias <code>island</code>), <code>asset_id</code>	Bearer API key; if the key is planet-scoped, <code>auth.planet</code> must match the body's <code>planet</code>	<code>isValidPlanet()</code> whitelist check on <code>planet</code> ; <code>asset_id</code> presence check; looked up with <code>.eq('id', asset_id)</code> (parameterized, not string-built SQL)
<code>/api/v1/assets/{planet}</code>	POST	Path: <code>planet</code> . Body: arbitrary JSON object (spread into the insert — <code>title</code> required)	Bearer API key; <code>planet</code> in path must match the key's scoped planet if set	<code>isValidPlanet()</code> on path param; <code>title</code> type/presence check; <code>account_id/created_by/updated_by</code> are overwritten server-side from <code>auth.accountId</code> after the body spread, so a caller cannot spoof ownership of the created asset
<code>/api/v1/assets/{planet}/{id}</code>	GET	Path: <code>planet, id</code>	Bearer API key; requires <code>verify</code> scope (<code>requireScope</code>) and matching planet scope (<code>requirePlanet</code>)	<code>isValidPlanet()</code> on <code>planet</code> ; <code>id</code> used only in a parameterized <code>.eq('id', id)</code> lookup; response uses an explicit allow-listed column list (<code>PUBLIC_ASSET_COLS</code>), never <code>select('*')</code> , so no internal field can leak via this path even if the row is found

Endpoint	Method	User-controlled input	Auth / authorization	Validation
/api/v1/assets/{planet}/{id}/history	GET	Path: planet, id	Bearer API key + scope/planet checks (same pattern as above)	Same as above
/api/v1/certificates/{number}	GET	Path: number (certificate number)	Bearer API key (no additional scope — global lookup by design, doc'd in the route as "no planet parameter needed")	certNumber used only in parameterized .eq('certificate_number', ...) lookups across a fixed, hardcoded list of tables — never interpolated into a query string
/api/v1/webhooks	POST	Body: outbound-webhook subscription config (URL, events, planet)	Bearer API key	Validated against the caller's own account scope before persisting

Note on this API's design as it relates to user input: every lookup uses the Supabase client's parameterized query builder (`.eq(...)`, `.in(...)`) — the path/body values are always passed as bind values, never concatenated into a raw SQL string, which is the primary control against SQL injection on this surface.

2. Internal Application API (src/app/api/**)

2.1 Resource-by-ID routes (dynamic path segment [id])

Endpoint	Method	User-controlled input	Auth / authorization	Validation
/api/operations/invoices/[id]/pdf	GET	Path: id	Signed-in user required; row ownership re-checked — the invoice query adds <code>.eq('account_id', user.id)</code> in addition to <code>.eq('id', id)</code> , and an active subscription is required	id is a bind parameter, not interpolated; 404 returned (not 403) if the row exists but isn't owned, avoiding existence-leak
/api/operations/loans/[id]/pdf	GET	Path: id	Same pattern as invoices	Same pattern
/api/operations/commitments/[id]/pdf	GET	Path: id	Same pattern as invoices	Same pattern

2.2 Search / lookup routes (query-string parameters)

All of the following are **GET** requests whose only input is one or more query-string parameters, generally used to build an **ILIKE** text-search filter. This group represents the largest "free text from the user" surface in the internal API and is where SQL/query-syntax injection risk would most likely appear if unguarded.

Endpoint	Query params	Auth	Validation / hardening
/api/search-artworks	q	Public (result set additionally scoped: signed-out sees <code>is_public=true</code> only; signed-in also sees own rows)	Zod (<code>min(2).max(100)</code>), <code>escapeLike()</code> escapes <code>%/_\</code> before building the <code>ilike</code> pattern, rate-limited (60/window)
/api/search-artists	q	Public	Same <code>escapeLike</code> + length-bounded pattern
/api/search-galleries	q	Public	Same pattern
/api/search-accounts	q, role	Public	Zod schema (q 2–100 chars, <code>role</code> ≤32 chars); <code>role</code> is then validated against a closed enum (<code>isValidRole</code>) rather than used raw in the query
/api/search-contacts	q	Signed-in (CRM contacts scoped to the caller's own account)	Length-bounded, escaped
/api/search-artwork-text	q, field	Signed-in, row-scoped	<code>field</code> constrained to a fixed allow-list of column names (never used to build a dynamical column reference from raw input)
/api/check-username	username	Public	Zod: 2–50 chars and <code>^[a-zA-Z0-9_.-]+\$</code> regex — rejects anything but the exact character set a username can contain, which also incidentally rules out SQL/ILIKE metacharacters; rate-limited (30/window)
/api/check-gallery-profile	(profile identifiers)	Signed-in	Ownership-scoped query
/api/artist-preview	accountId, profileId, posterAccountId (at least one required)	Public (returns only public profile/portfolio data)	Zod: each param must be <code>z.string().uuid()</code> — malformed/non-UUID values are rejected before ever reaching a query; rate-limited (120/window)
/api/artworks/feed	seed, offset, limit, q, sort	Public, with <code>following</code> sort gated to signed-in	Full Zod schema: <code>offset/limit</code> coerced+bounded integers (<code>limit</code> max 20), <code>sort</code> constrained to a closed enum (<code>shuffle recent top following exhibitions</code>) so it can never be used to inject an arbitrary <code>ORDER BY</code> /filter clause, <code>q</code> escaped via <code>escapeLike</code>
/api/get-user-exhibitions	userId	Signed-in — explicit IDOR check : <code>user.id !== userId → 403</code>	Straightforward equality check before any query runs
/api/check-exhibition-ownership	exhibitionId, userId	Signed-in — same explicit <code>user.id !== userId</code> check	Ownership additionally re-verified against the <code>exhibitions.gallery_id</code> column server-side (never trusts a client-asserted "I own this" claim)
/api/stripe/connect/status	(account context, session-derived)	Signed-in	Reads only the caller's own Stripe Connect status

2.3 Mutation / action routes (JSON body)

Endpoint	Method	Body input	Auth	Validation
/api/certificate-invite	POST	Invite recipient details, artwork/claim reference	Signed-in, ownership-checked	Zod-validated payload
/api/profiles/parse-input	POST	Free-text profile input (for AI-assisted parsing)	Signed-in	Size-bounded, passed to a constrained AI extraction task, not executed/evaluated

Endpoint	Method	Body input	Auth	Validation
/api/heartbeat	POST	none beyond session	Signed-in	Calls the <code>record_user_heartbeat</code> DB function, which itself enforces <code>auth.uid() = p_user_id</code> server-side (see principle-of-least-privilege.md §3)
/api/email/send	POST	Recipient/template/merge-field data	Signed-in / internal-only callers	Recipient and template resolved against allow-listed internal templates, not arbitrary attacker-supplied HTML
/api/admin/leads	GET, POST	Query: <code>datasetId</code> , <code>runId</code> ; Body: lead data	Admin only (require <code>AdminApi()</code>)	Full admin+MFA gate (see authentication-and-access-control.md §3.3) before any parameter is used
/api/admin/audio/denoise	POST	<code>multipart/form-data</code> audio file	Admin only	File-type/size checks before processing
/api/stripe/create-checkout-session	POST	<code>role</code> , <code>interval</code> , <code>priceId</code> selection	Signed-in	Requested <code>priceId</code> is cross-checked against the server's own known Stripe price IDs (from env config) — the client cannot submit an arbitrary price ID and have it silently accepted
/api/stripe/create-artwork-checkout-session	POST	<code>artworkId</code> , buyer details	Signed-in	Artwork looked up and price taken from the server-side row , not from client-submitted price data
/api/stripe/create-domain-checkout-session	POST	Domain name, plan	Signed-in	Domain format validated before a Stripe session references it
/api/stripe/connect/create-account, /api/stripe/connect/account-session	POST	Onboarding fields	Signed-in	Tied to the caller's own account; cannot create/link a Connect account on someone else's behalf
/api/stripe/create-portal-session	POST	none beyond session	Signed-in	Portal session created only for the caller's own Stripe customer id
/api/log-client-error	POST	Arbitrary client-reported JSON	Public (client-side error reporting)	Explicit key allow-list (<code>sanitizeClientErrorPayload</code>) — only <code>message/uploadedTotalImages/userAgent/platform/viewport/chunkCount</code> are extracted; every other field is silently dropped, strings are truncated to 500 chars before logging (prevents log injection / arbitrary-payload logging); rate-limited (30/window)
/api/onboarding/upload-cv	POST	<code>multipart/form-data</code> file (CV)	Signed-in	<code>assertAllowedFile</code> / <code>assertAllowedFileWithAv</code> check real file signature (not just the claimed <code>Content-Type</code>) against an allow-list (PDF/DOCX/DOC/TXT/CSV) and a 10 MB size cap, plus anti-virus scanning, before the file is stored or parsed
/api/taco/chat, /api/grants/chat, /api/opportunities/chat	POST	Free-text chat message + conversation history	Signed-in (subscription-gated)	User text is sent to the model only as conversation content , never concatenated into a server-side prompt that also contains privileged instructions in a way the user could override; all "tool" actions the AI can take (e.g. <code>handleCreateExhibition</code> , <code>handleUpdateArtistBio</code>) re-enter the same Supabase client bound to the caller's session, so the AI cannot act with more privilege than the signed-in user already has — see principle-of-least-privilege.md

2.4 Webhooks & scheduled (cron) routes — not user-facing, but network-reachable

Endpoint	Method	Input	How it's restricted
<code>/api/webhooks/stripe</code>	POST	Raw Stripe event payload (headers + body)	Stripe signature verified (<code>stripe.webhooks.constructEvent</code>) against <code>STRIPE_WEBHOOK_SECRET</code> before any field of the body is trusted — an attacker who merely POSTs to this URL without a valid signature is rejected outright
<code>/api/cron/stripe-reconcile</code>	GET, POST	none (scheduled trigger)	Authorization: Bearer <code><CRON_SECRET></code> checked via <code>constantTimeEquals()</code> (timing-safe compare), fails closed if <code>CRON_SECRET</code> is unset
<code>/api/cron/lifecycle-emails</code>	GET	none	Same <code>CRON_SECRET</code> pattern
<code>/api/cron/operations-alerts</code>	GET, POST	none	Same <code>CRON_SECRET</code> pattern

2.5 Auth callback routes (query parameters from redirect flows)

Endpoint	Method	Query params	Notes
<code>/auth/callback</code>	GET	OAuth <code>code</code> , <code>next</code> redirect path	Exchanged directly with Supabase Auth for a session (<code>exchangeCodeForSession</code>); the <code>next</code> redirect target is constrained to same-origin relative paths
<code>/auth/confirm</code>	GET	Supabase <code>token_hash</code> , <code>type</code> , <code>next</code>	Verified via <code>supabase.auth.verifyOtp()</code> — the token itself is the security boundary, not the URL structure

2.6 Session-context-only routes (no meaningful user-supplied parameter)

For completeness: `/api/admin/check` (GET) and `/api/check-gallery-profile` take no query/body input beyond the caller's own session cookie — included here to make clear the inventory above is exhaustive of the route list, not a filtered subset.

3. Dynamic page routes (URL path segments)

These are Next.js pages (not JSON APIs) whose URL contains a user-suppliable segment. Listed because the question asks about "portions of the API/**URL**." None of these execute raw SQL from the path segment — each resolves to a Postgres lookup that is additionally constrained by Row Level Security (see [authentication-and-access-control.md](#) and [principle-of-least-privilege.md](#) for the full RLS/least-privilege model that governs every one of these lookups).

Route pattern	Path segment(s)	What it resolves to
<code>/artworks/[id]</code>	<code>id</code>	A single artwork (RLS: owner or <code>status='verified'/is_public</code>)
<code>/artists/[id], /profiles/[id]</code>	<code>id</code>	A public profile
<code>/collectibles/[id]</code>	<code>id</code>	A collectible asset (same visibility rules as artworks)
<code>/exhibitions/[id]</code>	<code>id</code>	A published exhibition
<code>/grants/proposals/[id]</code>	<code>id</code>	A grant proposal (author/collaborator scoped)

Route pattern	Path segment(s)	What it resolves to
/gallery/[name], /gallery/[segment], /g/[slug]	slug/name	A gallery's public page/short link
/blog/[slug], /open-calls/[slug]	slug	Published content lookup by slug
/_sites/[handle], /_sites/[handle]/works/[artworkId], /_sites/[handle]/exhibitions/[id]	handle, id	Creator-site tenant routing (resolved in <code>src/middleware.ts</code> , see <code>authentication-and-access-control.md</code> §3.1)
/admin/blog/[id]	id	Admin-only content edit (behind <code>requireAdmin()</code> , see the admin layout gate)
/investors/[doc], /portal/market-cap/[scope], /docs/[...slug], /docs/api/[...slug]	doc/scope/slug	Static/curated content selection, not arbitrary file-path traversal (Next.js route matching, not filesystem <code>readFile(path)</code>)

4. Common input-validation controls applied across the surface

Control	Where used	Purpose
Zod schema validation (<code>z.object()</code> , <code>.uuid()</code> , <code>.enum()</code> , length bounds)	Most query/body-parsing routes (<code>search-*</code> , <code>check-username</code> , <code>artworks/feed</code> , <code>artist-preview</code> , etc.)	Rejects malformed/oversized/wrong-type input before it reaches any database call
<code>escapeIlike()</code> (<code>src/lib/escape-ilike.ts</code>)	Every free-text search endpoint that builds an <code>ilike</code> pattern	Escapes <code>%</code> , <code>_</code> , and <code>\</code> so user text cannot widen or hijack the intended wildcard pattern
Closed enum / allow-list params (e.g. <code>sort</code> , <code>field</code> , <code>role</code>)	<code>artworks/feed (sort)</code> , <code>search-artwork-text (field)</code> , <code>search-accounts (role)</code> , planet routes (<code>isValidPlanet</code>)	Prevents a parameter from being used to reach an unintended code path or column, regardless of what string is supplied
Parameterized queries only	Every route (Supabase client <code>.eq().in().or()</code> builders) — no route in this codebase concatenates user input into a raw SQL string	Standard SQL-injection defense; confirmed by inspection, not just convention
Ownership re-checks beyond RLS	<code>get-user-exhibitions</code> , <code>check-exhibition-ownership</code> , <code>operations/*/pdf</code> routes	Defense-in-depth IDOR prevention — explicit <code>callerId === resourceOwnerId</code> checks in code, in addition to the database's own RLS policies
Rate limiting (<code>src/lib/rate-limit.ts</code> , Upstash-backed with in-memory fallback)	All public search/lookup endpoints, <code>log-client-error</code> , <code>check-username</code> , <code>artist-preview</code>	Bounds abuse of unauthenticated, input-accepting endpoints (scraping, brute force, DoS)
File-signature + AV validation (<code>src/lib/file-signature.ts</code>)	<code>onboarding/upload-cv</code> and other file-upload routes	Verifies actual file bytes match an allow-listed type (not just the client-asserted <code>Content-Type</code>), enforces a size cap, and scans for malware before storage
Key allow-listing for logged/echoed input	<code>log-client-error</code>	Prevents a client from injecting arbitrary fields/log lines through a "telemetry" endpoint
Signature verification for inbound webhooks	<code>webhooks/stripe</code>	Rejects any payload not cryptographically signed by the claimed sender
Constant-time secret comparison	<code>cron/*</code> routes	Prevents timing side-channel recovery of <code>CRON_SECRET</code>
API-key scope + planet binding	All <code>apps/api v1</code> routes	A key issued for one partner/vertical cannot use its <code>planet/id</code> path parameters to reach another partner's data

5. Summary

Every endpoint that accepts a user-suppliable URL segment, query parameter, or request body has been enumerated above, grouped by input shape. Across the whole surface, no route trusts a user-controlled value to determine *whose* data is returned without an independent server-side check (session identity, RLS, or an explicit ownership comparison), and no route builds a raw SQL/ILIKE string without escaping or a closed enum. The public partner API ([apps/api/v1](#)) additionally binds every request to an API key with its own scope and planet restriction, so path parameters cannot be used to pivot outside the caller's authorized data set.